HUMAIN HACKATHON CHALLENGE 1: AGENTIC RECOVERY OF A FAILING SYSTEM

Sentinel-RX

A security-aware AI agent that identifies, diagnoses, recovers and verifies the recovery of a failing system, even when several faults hit at once.

100%
recovered on unseen incidents, vs 45% for a rule-based runbook

0%
false “recovered” claims, vs 70% for a single-shot LLM

0
harmful actions, on a fully local 7B model

Abdulrahman Alharbi · abdulalharbi.github.io/sentinel-rx

About me



Abdulrahman Alharbi

Computer scientist, AI & cybersecurity
Riyadh, Saudi Arabia

alharbi.ams@outlook.com
LinkedIn

Education

M.S. Computer Science, UNLV (2026), GPA 3.92, specializing in AI and cybersecurity

B.S. Computer Science, Information Assurance, UNLV (2025)

Research

Master's thesis: periodicity-based triage of APT command-and-control beaconing in encrypted network flows with machine learning

AP 0.852 vs 0.046 baseline; up to 99% less analyst workload.

Experience

Software Engineer in Test, UNLV Office of Information Technology

Test automation and CI/CD across 3 app versions; helped adopt enterprise generative AI in engineering

Certifications

NSA CAE-CD: Information Assurance

AWS Academy Cloud Foundations
SDAIA: Fundamentals of Artificial Intelligence

SDAIA: AI Concepts and Advanced Applications

The problem: failures rarely come alone

Faults stack and confound

A saturated database pool makes the API look broken. Fix the symptom and the cause stays.

Attacks hide inside outages

Restarting a host that carries a command-and-control implant destroys evidence and re-infects it. Ops tooling doesn't check.

"Recovered" is claimed, not proven

On our unseen test incidents, a single-shot LLM declared recovery falsely in 70% of runs.

Why it matters

Local
Saudi enterprises and government systems need AI operations that keep logs and security data inside the organization, on sovereign infrastructure.

Global
Incident response is one of the most expensive, least automated parts of running software, and systems grow faster than on-call teams.

Research first: five gaps in the literature

My structured literature review of AIOps, LLM agents, agentic SRE benchmarks, remediation safety and C2 beacon detection [1] found that agents are now tested on live, fault-injected systems, but success on realistic benchmarks stays low [4, 5] and evaluations skip five things:

G1

Concurrent faults

Most tests inject one fault.

1–3 faults per incident; recovery stays 100% at every level

G2

Security-blind ops

No AIOps benchmark checks for a hidden compromise.

Beacon detector + contain-before-restart; 0 harmful actions

G3

Unverified recovery

False "recovered" is rarely measured.

False claims as a headline metric: 0% vs 70%

G4

Agent cost

Tokens and compute rarely reported.

6,103 tokens per incident on a local 7B model

G5

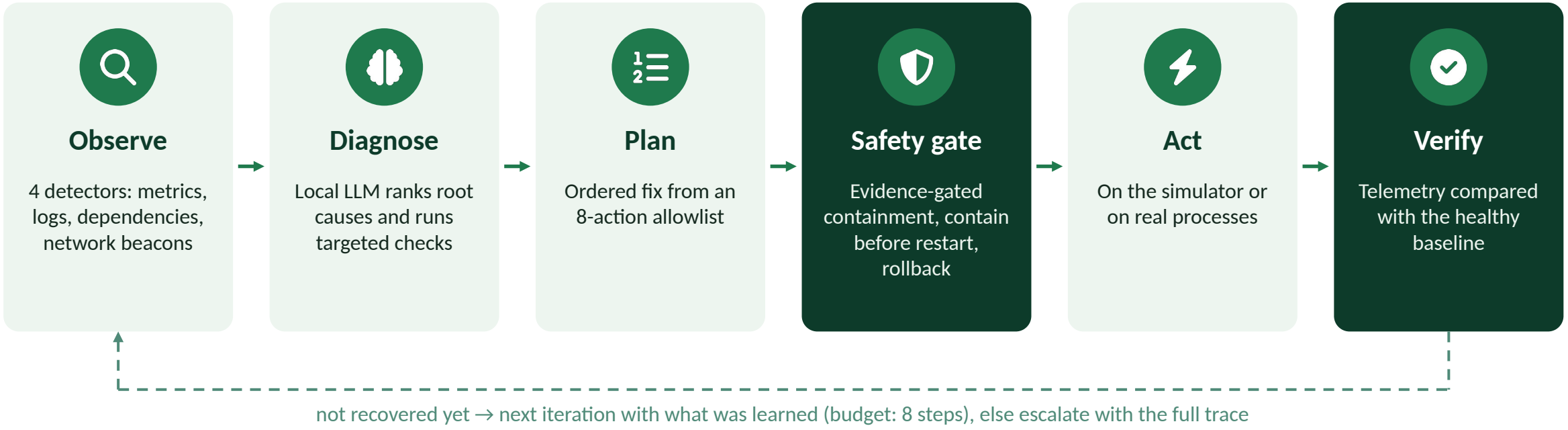
Robustness

Little testing on degraded telemetry.

95–100% recovery under four perturbations

Grey: the gap in the literature. Green: how Sentinel-RX answers and measures it. The beacon detector builds on my master's thesis [2].

The solution: a verified recovery loop



Design principle The LLM reasons over compact evidence and never touches raw telemetry. Deterministic components make it safe (the safety gate) and honest (the verification gate): "recovered" is only reported when the telemetry confirms it.

How it’s built

AI model	qwen2.5:7b via Ollama, local, temperature 0, schema-validated JSON
Detectors	Robust z-scores, log templates, dependency ranker, RandomForest beacon detector (AP 0.999 on a synthetic test set)
Environments	Simulator (5 services, 10 fault types) and a live system of 5 real processes
Benchmark	30 dev / 20 held-out scenarios, 5 perturbations, 5 baselines, 4 ablations
Engineering	Python 3.12, 109 tests, CI, record/replay cache for every LLM call
Footprint	One laptop: no cloud, no Docker, no GPU needed to replay results

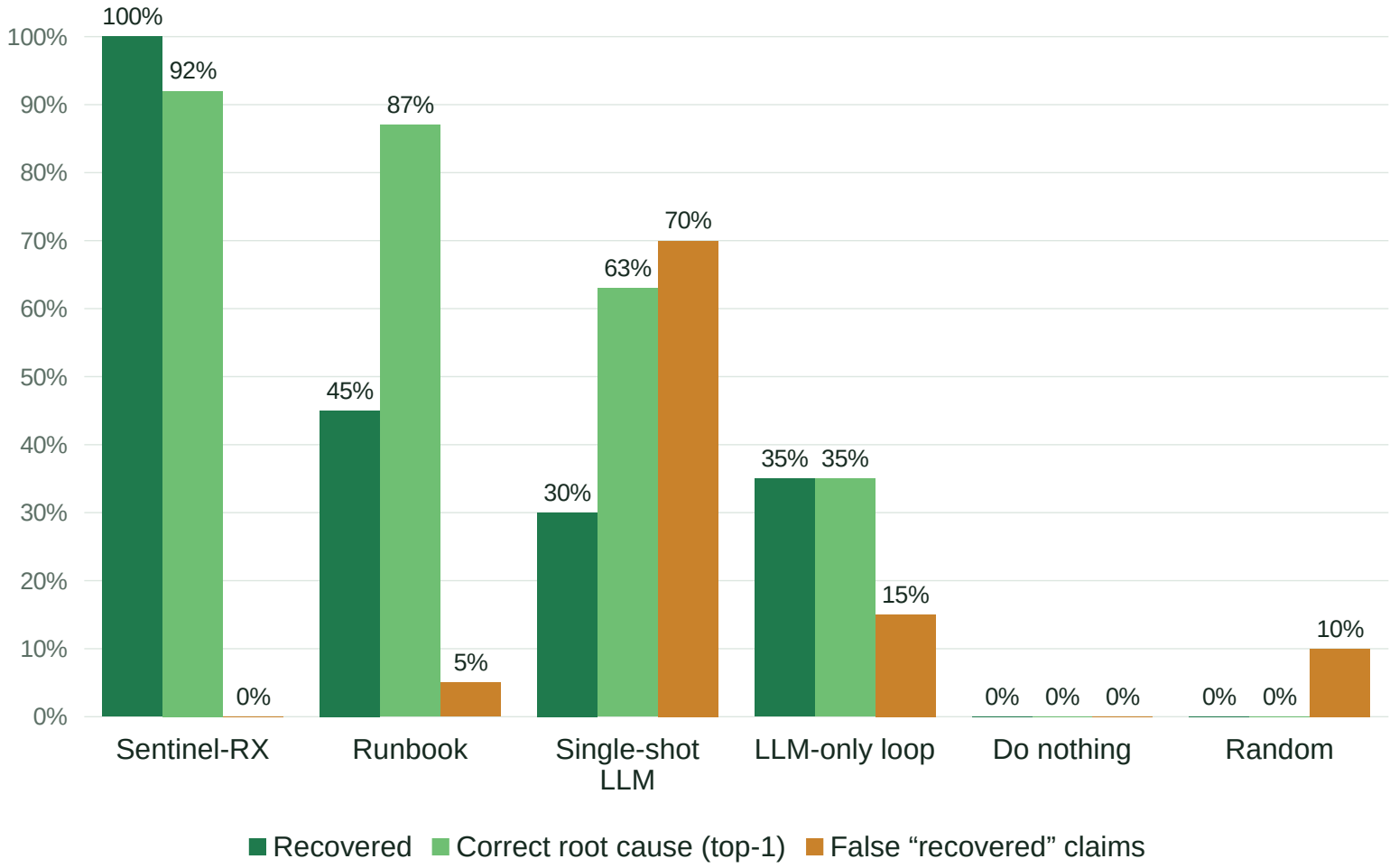
Live demo on real processes

3 concurrent faults: memory leak (api), connection-pool exhaustion (db), C2 beacon (worker). Health falls from 1.00 to 0.10.

1. Model proposes restarting the worker. Safety gate blocks it: beacon open, contain first.
2. Fixes db and api; verifier refuses “recovered” while the beacon persists.
3. Blocks worker egress, then restarts it. Verified recovered, health 1.00.

6 LLM calls · 9,058 tokens · restart before containment: 0

Results on incidents it had never seen



100% recovered
95% CI 100–100

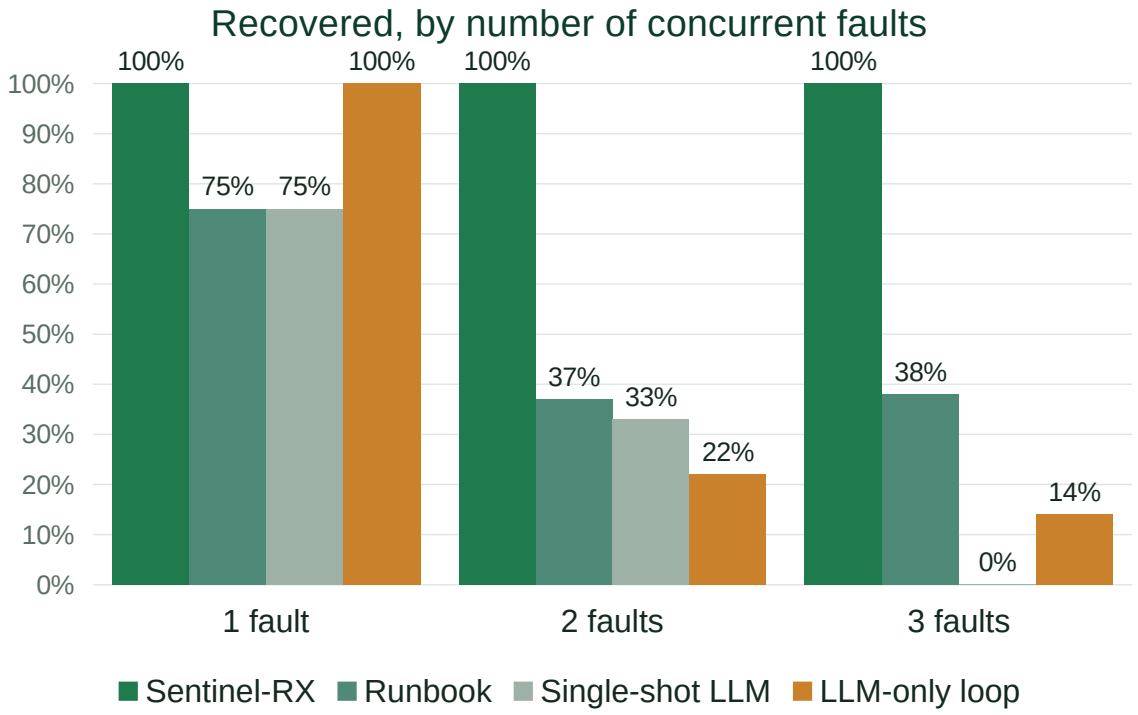
92% correct root cause
per-fault recall 82%

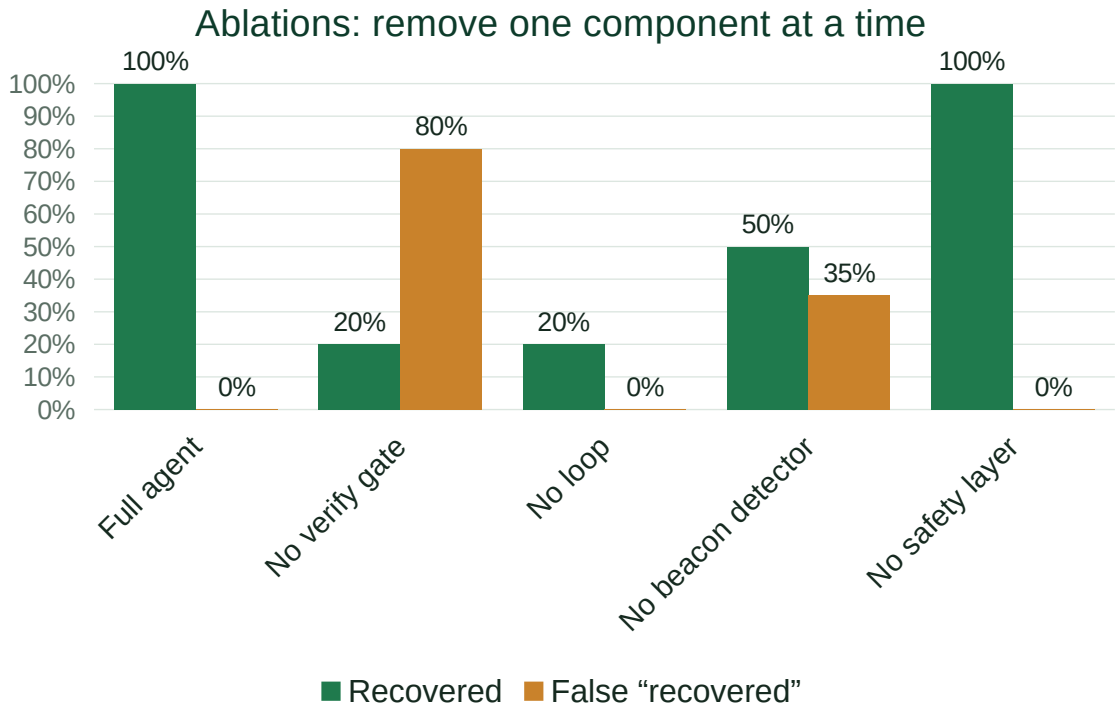
0% false “recovered”
single-shot LLM: 70%

6.1k tokens per incident
LLM-only loop: 35k for 35%

Held-out split: 20 scenarios never used during development × 3 seeds = 60 runs per configuration (LLM-only loop: 1 seed, 20 runs). Agent frozen before the first held-out run. 95% bootstrap confidence intervals in the repo.

Robust, and every component earns its place





Without the safety layer recovery is unchanged, but 0.60 harmful actions per run appear.

Perturbations
noisy logs 100% · missing metrics 95% · renamed services 100% · timestamp jitter 100% recovered

Honest evaluation
Frozen before held-out; mid-project scorer audit; do-nothing and random floors recover 0%

Reproducible
CI replays all 508 committed held-out and perturbation runs; any difference fails

What makes it different

Security-aware recovery

Finds C2 beacons hidden among normal traffic and contains a compromised host before restarting it.

Verified, not believed

Recovery is only reported when telemetry confirms it: 0% false claims vs 70% for a single-shot LLM.

Sovereign and efficient

Runs on a laptop with a local 7B model: 6,103 tokens per incident vs 35,018 for an LLM reading raw telemetry.

Reproducible by design

Every number replays from a committed cache; CI re-checks all of them on every change.

Potential impact

Local
Sovereign AI operations for Saudi enterprises and government: no logs leave the organization, and it fits on-premises and national AI infrastructure.

Global
An open benchmark for concurrent, security-aware recovery that measures false claims, harm and cost, not just success.

1 laptop · 0 cloud calls · 6.1k tokens/incident

Challenges, limits and what's next

Challenges solved

- A 7B model mixed up services → per-service evidence and a repeated-plan guard
- The recovery metric was too lenient → mid-project scorer audit with sanity baselines
- One test split did not replay → stable hashing and a CI replay check

Honest limitations

- Small, hand-written simulator; the prompt knows each fault's fix, as a runbook does
- An unseen fault type is recovered and localized, but not named
- Beacon model trained on synthetic data; secondary tests use one seed

Next steps

- Real telemetry: Prometheus metrics, Zeek/NetFlow, log stores
- Retrain the beacon detector on real captures
- Human approval for irreversible actions; evaluate sovereign models such as ALLaM

Conclusion & demo

- Identifies, diagnoses, recovers and verifies, even with three concurrent faults and a hidden attack
- On unseen incidents: 100% recovered, 0% false claims, 0 harmful actions
- Fully local and fully reproducible: one command replays every number

Live site, results and incident replay: abdulalharbi.github.io/sentinel-rx
Source code: github.com/AbdulAlharbi/sentinel-rx
Technical brief (6 pages) [3]: sentinel_rx_brief.pdf
Full technical report (27 pages) [3]: sentinel_rx_technical_report.pdf
Literature review [1]: literature_review.pdf

Scan for the live demo site

[1] A. Alharbi, “Agentic Diagnosis and Recovery of Failing Systems: A Structured Literature Review of AIOps, LLM Agents, and Security-Aware Operations,” Oct. 2026.
[2] A. Alharbi, “Periodicity-Based Triage of APT Command-and-Control Beaconing in Network Flows Using Machine Learning,” M.S. thesis, UNLV, 2026.
[3] A. Alharbi, “Sentinel-RX: technical brief and technical report,” Oct. 2026.
[4] S. Jha et al., “ITBench: Evaluating AI agents across diverse real-world IT automation tasks,” ICML, 2025. [5] J. Xu et al., “OpenRCA: Can large language models locate the root cause of software failures?” ICLR, 2025.